

Report secondo progetto di Metodi del calcolo scientifico

Daniele Besozzi 894998
Andrea Consonni 900116
Matteo Cervini 902225

29 maggio 2026

Indice

1	Prima parte: implementazione e benchmark DCT2	2
1.1	Metodologia di misura delle performance	2
1.2	Risultati	3
2	Seconda parte: compressione immagini	5
2.1	Struttura del progetto	5
2.2	Test su immagini	5
2.2.1	Tony Pitony	5
2.2.2	Lena	8

Capitolo 1

Prima parte: implementazione e benchmark DCT2

Per l'implementazione della DCT2 abbiamo scelto il linguaggio di programmazione Java, mettendo a confronto la nostra implementazione con l'implementazione fornita dalla libreria `JTransform wendykier_jtransforms_3_`. Per effettuare le operazioni di algebra lineare richieste in modo semplice ed ottimale, abbiamo scelto di utilizzare la libreria `Efficient Java Matrix Library(ejml)abeles_ejml_0_44_0`.

1.1 Metodologia di misura delle performance

Per misurare le performance della nostra implementazione rispetto a quella della libreria, abbiamo generato in maniera causale matrici di dimensioni pari a 2^i , $i = 3, \dots, 13$. Le entrate di questa matrice sono generate in maniera casuale in un intervallo di valori a virgola mobile compreso tra 0 e 100.

```
1  public static double[][] randomMatrix(int n) {
2
3      double[][] m = new double[n][n];
4
5      for (int i = 0; i < n; i++) {
6          for (int j = 0; j < n; j++) {
7              m[i][j] = Math.random()*100;
8          }
9      }
10
11     return m;
12 }
```

Listing 1.1: Generazione di matrici casuali

Per eseguire le misurazioni, abbiamo utilizzato la libreria `Java Microbenchmark Harness(JMH)openjdk_jmh`, che ci ha permesso di eseguire i benchmark in modo affidabile e preciso, gestendo automaticamente le iterazioni di warmup e misurazione. Poiché Java esegue ottimizzazioni a runtime (JIT compilation), è necessario eseguire iterazioni di warmup prima delle misurazioni effettive. Durante la fase di warmup, il codice viene compilato e ottimizzato dalla JVM (Java Virtual Machine) `ibm_jit_optimization`. Successivamente, eseguiamo molteplici run e calcoliamo la media dei tempi ottenuti da queste iterazioni, scartando i risultati della fase di warmup. Il codice di configurazione del benchmark è riportato nel listing 1.2.

```

1  public double run(Supplier<Supplier<?>> taskFactory) throws Exception {
2
3      Options opt = new OptionsBuilder()
4          .include(String.format(BenchmarkConstants.JMH_BENCHMARK_INCLUDE_TEMPLATE,
5              BenchmarkRunner.class.getSimpleName()))
6          .forks(0)
7          .warmupIterations(DEFAULT_WARMUP_ITERATIONS)
8          .measurementIterations(DEFAULT_MEASUREMENT_ITERATIONS)
9          .build();
10
11     Collection<RunResult> results;
12     try {
13         pendingFactory = taskFactory;
14         results = new Runner(opt).run();
15     } finally {
16         pendingFactory = null; // always clear to prevent stale lambda on reuse
17     }
18
19     if (results.isEmpty()) {
20         throw new CancellationException(BenchmarkConstants.BENCHMARK_ABORTED_NO_RESULTS);
21     }
22     return results.iterator().next().getPrimaryResult().getScore();
23 }

```

Listing 1.2: Configurazione del benchmark con JMH

1.2 Risultati

I risultati del benchmark sono riportati nella figura 1.1. Come si può evincere dal grafico, i tempi di

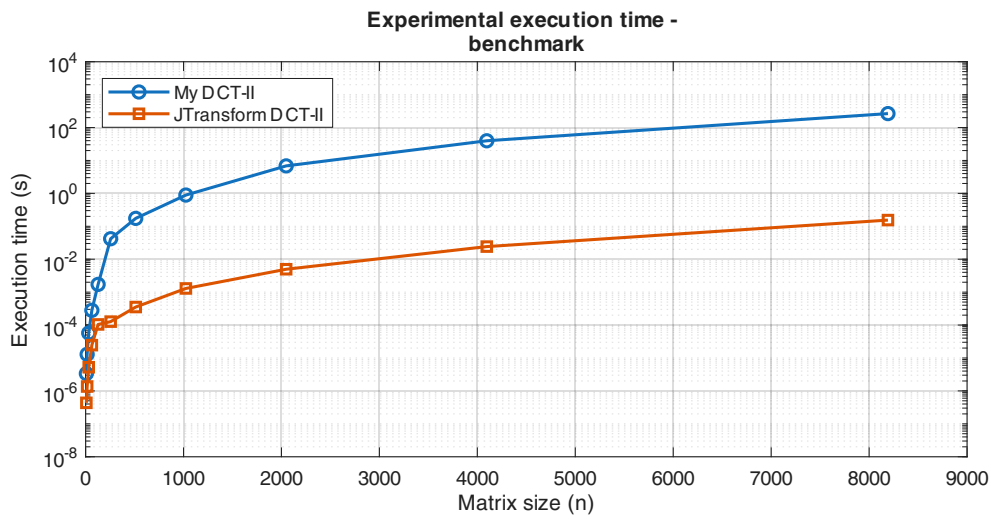


Figura 1.1: Risultati del benchmark tra la nostra implementazione della DCT2 e quella fornita da JTransform

esecuzione dell'implementazione fornita dalla libreria sono di gran lunga inferiori rispetto alla nostra implementazione. L'algoritmo implementato dalla libreria è nella versione fast (FFT) e implementa una variante dell'algoritmo dividi-et-impera per la FFT (split-radix e mixed-radix) ottimizzato per sistemi multiprocessore `jtransforms_dct2d`. Ciò comporta un miglioramento significativo delle prestazioni

dato sia dall'algoritmo utilizzato sia dalla capacità della libreria di sfruttare le capacità di multithreading del processore.

Capitolo 2

Seconda parte: compressione immagini

2.1 Struttura del progetto

2.2 Test su immagini

Per provare la nostra implementazione della compressione tramite DCT2, abbiamo scelto due immagini: una di Tony Pitony e una di Lena (immagine classica per l'immagine processing). Per entrambe le immagini, abbiamo applicato la DCT2 con diversi valori di f (la dimensione del blocco) e d (il numero di coefficienti da mantenere). Di seguito sono riportati i risultati ottenuti per ciascuna immagine, le quali sono anche disponibili in formato .bmp al [link](#).

2.2.1 Tony Pitony

Per quanto riguarda l'immagine in figura 2.1 abbiamo provato a modificare la dimensione dei blocchi f e il coefficiente del taglio d . I valori di f che abbiamo scelto sono stati 8, 16, 32, 64 e 128, mentre i valori di d sono stati rispettivamente 4, 8, 16, 32 e 64, ovvero la metà della dimensione del blocco. Fare benchmark aumentando la dimensione dei blocchi può risultare interessante per questa immagine perché contiene sia aree uniformi sia dettagli ad alta frequenza, come i bordi degli occhiali e la texture della camicia. Blocchi più grandi migliorano generalmente la compressione nelle zone lisce, ma possono introdurre artefatti più evidenti e perdita di dettaglio nelle regioni ricche di dettagli.



Figura 2.1: Immagine di Tony Pitony non compressa



Figura 2.2: Immagine di Tony Pitony compressa con DCT2, con $f = 8$ e $d = 4$



Figura 2.3: Immagine di Tony Pitony compressa con DCT2, con $f = 16$ e $d = 8$



Figura 2.6: Immagine di Tony Pitony compressa con DCT2, con $f = 128$ e $d = 64$



Figura 2.4: Immagine di Tony Pitony compressa con DCT2, con $f = 32$ e $d = 16$



Figura 2.5: Immagine di Tony Pitony compressa con DCT2, con $f = 64$ e $d = 32$

All'aumentare della dimensione dei blocchi, possiamo vedere che la qualità dell'immagine compressa peggiora particolarmente nelle aree con molti dettagli. In particolare, si può anche notare l'introduzione di artefatti di tipo ringing nelle zone uniformi, come la finestra, che diventano più evidenti con blocchi più grandi. Un esempio di questi artefatti è mostrato in figura 2.7, dove si possono vedere delle bande scure e chiare intorno ai bordi, tipiche del fenomeno di ringing. L'effetto sulle nostre immagini è evidenziato in figura 2.8, dove si possono notare le bande scure attorno alla gamba e ai bordi della finestra.



Figura 2.7: Artefatti di tipo ringing



Figura 2.8: Zoom sull'immagine di Tony Pitony compressa con DCT2, con $f = 128$ e $d = 64$, evidenziando gli artefatti di tipo ringing

2.2.2 Lena

Per quanto riguarda l'immagine in figura 2.9 abbiamo provato a mantenere fissa la dimensione dei blocchi f a 8 e a modificare il coefficiente del taglio d con i valori 12, 8, 4, 2 e 0. In particolare, con $d = 0$ abbiamo eliminato tutti i coefficienti, ottenendo così un'immagine completamente nera. Si può osservare che, come ci si aspetta, all'aumentare del numero di coefficienti mantenuti, la qualità dell'immagine compressa migliora, con una maggiore fedeltà all'immagine originale. In particolare, con $d = 12$ e $d = 8$ l'immagine compressa è molto simile all'originale, mentre con $d = 4$ si notano perdite di dettagli e si manifesta il tipico effetto di jpeg dove sono ben visibili i blocchi di dimensione 8×8 .



Figura 2.9: Immagine di Lena non compressa



Figura 2.10: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 12$



Figura 2.11: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 8$



Figura 2.12: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 4$



Figura 2.13: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 2$

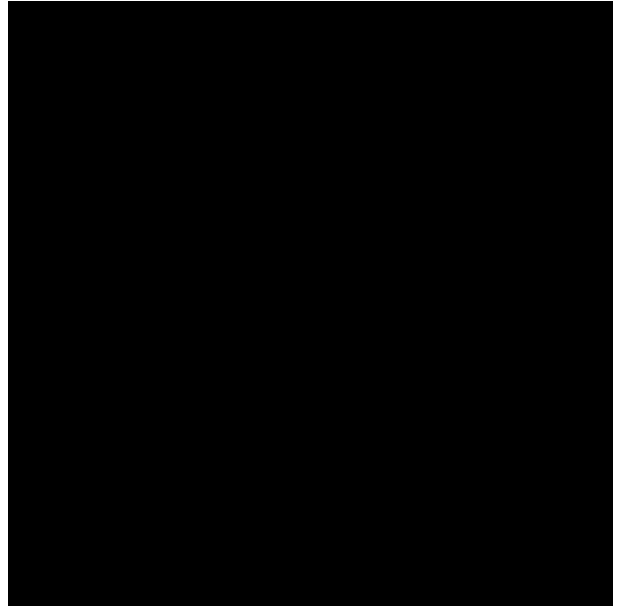


Figura 2.14: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 0$